



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Instituto de Investigaciones en Matemáticas Aplicadas y
Sistemas

Práctica 5 - Agrupamiento y clasificación

PRESENTA

Munive Ramírez Ibrahim

PROFESORA

Dra. María Elena Martínez Pérez

ASIGNATURA

Reconocimiento de Patrones

1. Objetivos

Esta práctica introduce el modelo de bolsa de palabras o *Bag of Visual Words* (BoVW) para la clasificación de imágenes. El flujo de trabajo incluye:

- Extraer descriptores SIFT de un conjunto de imágenes.
- Construir un vocabulario visual aplicando k-means.
- Representar cada imagen mediante un histograma normalizado de palabras visuales.
- Clasificar las imágenes.
- Evaluar el modelo con exactitud y matriz de confusión.
- Experimentar con distintos parámetros (número de palabras, tipo de clasificador, etc.).

2. Introducción

El reconocimiento de patrones en imágenes es un área fundamental de la visión por computadora. Uno de los enfoques más populares para la clasificación de escenas u objetos es el modelo *Bag of Words* (BoVW), inspirado en el procesamiento de texto. En lugar de palabras, se utilizan *palabras visuales* obtenidas a partir de descriptores locales de las imágenes. Este método es robusto frente a variaciones de escala, rotación e iluminación parcial.

2.1. Bolsa de palabras (Bag of Words)

El modelo (BoW) es ampliamente utilizado en procesamiento de lenguaje natural. Consiste en representar un documento como un vector de frecuencias de palabras de un vocabulario predefinido, ignorando el orden de las palabras pero preservando la información semántica relevante. Por ejemplo, si el vocabulario es $\{\textit{perro}, \textit{gato}, \textit{ratón}\}$, un documento con las palabras “gato gato perro” se representa como $[1, 2, 0]$.

La clave del éxito de BoW es que palabras similares tienden a aparecer en contextos similares, y dos documentos con distribuciones de palabras parecidas probablemente tratan sobre el mismo tema.

2.2. Traslación al dominio visual: palabras visuales

De manera análoga, en el enfoque BoW se considera que una imagen es un “documento” compuesto por “palabras visuales”. Estas palabras no son objetos completos, sino **parches locales de la imagen**

que han sido cuantificados en un vocabulario. El proceso típico consta de tres fases:

1. **Extracción de descriptores locales:** Se detectan regiones de interés (por ejemplo, esquinas, bordes, texturas) y se calcula un descriptor numérico que resume la apariencia de cada región. Uno de los descriptores más potentes y ampliamente usados es **SIFT** (Scale-Invariant Feature Transform), desarrollado por David Lowe en 2004. SIFT es invariante a rotación y escala parcial, y robusto a cambios de iluminación. Cada descriptor es un vector de 128 números reales.
2. **Construcción del vocabulario visual:** Se toman todos los descriptores extraídos de un conjunto de entrenamiento y se agrupan mediante un algoritmo de clustering, típicamente *k-means*. Cada centroide resultante define una “palabra visual”. El número k de palabras es un hiperparámetro crítico: valores pequeños producen palabras muy generales, valores grandes producen palabras muy específicas (riesgo de sobreajuste).
3. **Representación de la imagen como histograma:** Para una imagen nueva, se extraen sus descriptores SIFT y cada uno se asigna a la palabra visual más cercana (en distancia euclídea). Se cuenta cuántas veces aparece cada palabra y se normaliza el histograma para que sume 1. Este histograma es el vector de características que se usará para la clasificación.

2.3. Ventajas del modelo BoW

- **Invariancia parcial:** Al depender de descriptores locales invariantes (SIFT), el modelo tolera cambios de escala, rotación y traslación de los objetos.
- **Compacidad:** Una imagen se reduce de miles de píxeles a un histograma de dimensión k (típicamente entre 100 y 1000), lo que facilita el entrenamiento de clasificadores.
- **Simplicidad conceptual:** La analogía con el texto es fácil de entender y extender.
- **Buen rendimiento:** En su época, BoW logró resultados competitivos en benchmarks como Caltech-101.

2.4. Clasificador empleado: Máquina de Vectores Soporte (SVM)

Una vez que tenemos los histogramas, necesitamos un clasificador que aprenda a separar las distintas categorías. Utilizaremos una **Máquina de Vectores Soporte** con kernel lineal. El SVM busca el hiperplano que maximiza el margen entre las clases. Es especialmente eficaz cuando el número de características es moderado (nuestro k) y hay un número suficiente de ejemplos. La elección del kernel lineal evita la complejidad de kernels no lineales y suele funcionar bien para histogramas normalizados.

2.5. Métricas de evaluación

Para medir el rendimiento del clasificador usaremos dos herramientas estándar:

- **Accuracy** (exactitud): Proporción de aciertos global. Se define como $\frac{TP+TN}{TP+TN+FP+FN}$, aunque en clasificación multiclase se calcula como el número de predicciones correctas dividido entre el total. Es una medida intuitiva pero puede ser engañosa si las clases están desbalanceadas.
- **Matriz de confusión**: Una tabla de tamaño $C \times C$ (donde C es el número de clases) donde la entrada (i, j) indica cuántas muestras de la clase real i fueron clasificadas como clase j . Permite identificar confusiones específicas: por ejemplo, que muchos aviones sean etiquetados como pájaros, o que los coches y las motos se confundan entre sí.

3. Desarrollo

Ejercicio: Implementación de BoW y comparar resultados.

Selecciona uno de los tres conjuntos de datos para la implementación del modelo variando la configuración de sus hiperparámetros. Se sugiere utilizar un subconjunto de **Caltech-101** (disponible en `torchvision`) con tres categorías:

- `airplane` (avión).
- `car_side` (coche de perfil).
- `motorbikes` (motocicletas).

Alternativamente, para probar con más clases o imágenes propias:

- **CIFAR-10**: 10 clases de objetos pequeños (requiere redimensionar a 128×128).
- **Oxford Flowers-102**: 102 tipos de flores.
- **Dataset propio**: imágenes organizadas en carpetas por categoría.

1. El flujo de trabajo implementado en el script es el siguiente:

- a) Descargar y filtrar el dataset (3 clases, imágenes redimensionadas a 128×128).
- b) Extraer descriptores SIFT de todas las imágenes.

- c) Ejecutar k-means con k clusters para obtener el vocabulario.
- d) Para cada imagen, construir su histograma de palabras visuales (Mostrar 5).
- e) Dividir el conjunto en entrenamiento (80 %) y prueba (20 %).
- f) Entrenar un SVM lineal con los histogramas de entrenamiento utilizando búsqueda de hiperparámetros (*grid search*).
- g) Predecir las etiquetas del conjunto de prueba y calcular exactitud, comenta tus resultados.
- h) Mostrar la matriz de confusión, comenta tus resultados.

2. Evaluación y comparación de resultados:

- a) **Variar el número de palabras visuales** ($k = 50, 200, 500$). ¿Cómo afecta la exactitud y el tiempo de cómputo?
- b) **Cambiar el clasificador**: sustituir SVM por Random Forest y comparar resultados.
- c) **Agregar validación cruzada**: hacer validación cruzada con 5 particiones e informar la media y desviación estándar de la exactitud.

3. Opcional

- a) **Utilizar una base de datos diferente** y repetir la implementación del modelo adaptando la carga de datos.

4. Preguntas:

- ¿Por qué es necesario normalizar los histogramas?
- ¿Qué ocurre si k es demasiado pequeño? ¿y si es demasiado grande?
- ¿Podría utilizarse este método para detección de objetos en lugar de clasificación?

4. Código

4.1. Repositorio

Los notebooks .ipynb así como los demás archivos de la práctica se encuentran en el siguiente repositorio de GitHub: <https://github.com/Garp02/Reconocimiento-Patrones/tree/main/P5>

5. Conclusiones

6. Referencias

- D. G. Lowe, "Distinctive image features from scale-invariant keypoints", IJCV 2004.
- G. Csurka et al., "Visual categorization with bags of keypoints", ECCV 2004.
- Scikit-learn documentation: <https://scikit-learn.org>
- OpenCV Python tutorials: <https://docs.opencv.org>